arXiv:2512.20957v2 [cs.SE] 25 Dec 2025

One Tool Is Enough: Reinforcement Learning for Repository-Level LLM Agents

Zhaoxi Zhang [1] **Yitong Duan** [2] **Yanzhi Zhang** [2] **Yiming Xu** [1] **Jiyan He** [2] **Yunfang Wu** [1]

Abstract

Locating the files and functions requiring modification in large open-source software (OSS) repositories is challenging due to their scale and structural complexity. Existing large language model (LLM)-based methods typically treat this as a repository-level retrieval task and rely on multiple auxiliary tools, which overlook code execution logic and complicate model control. We propose **RepoNavigator**, an LLM agent equipped with a **single execution-aware tool**—jumping to the definition of a invoked symbol. This unified design reflects the actual flow of code execution while simplifying tool manipulation. RepoNavigator is **trained end-to-end via Reinforcement Learning (RL)** directly from a pretrained model, without any closed-source distillation. Experiments demonstrate that RL-trained RepoNavigator achieves state-of-the-art performance, with the 7B model outperforming 14B baselines, the 14B model surpassing 32B competitors, and even the 32B model exceeding closed-source models such as Claude-3.7. These results confirm that integrating **a single, structurally grounded tool with RL training** provides an efficient and scalable solution for repository-level issue localization.

1. Introduction

With the rapid advancement of Large Language Models (LLMs) (Liu et al., 2024; Team, 2024; Yang et al., 2025a), equipping LLMs with pre-built tools to form LLM agents has become a common paradigm for expanding their capabilities (Shen, 2024; Yuan et al., 2024; Lu et al., 2024). In the domain of software engineering (SWE), although LLM agents can effectively handle simple programming tasks (Hui et al., 2024; Guo et al., 2024a), their ability to operate on large-scale open-source software (OSS) repositories remains limited. SWE-BENCH (Jimenez et al., 2023) currently serves as the most comprehensive benchmark for evaluating whether LLMs can resolve real-world GitHub issues. All pretrained LLMs can not process the whole repository directly due to context limits. While SWE-AGENT (Jimenez et al., 2023) provides moderate gains, it remains far from enabling robust repository-level reasoning.

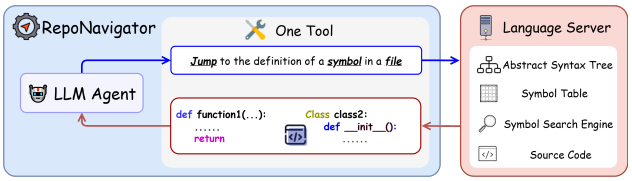

Figure 1. Illustration of a LLM navigating through a code repository. The LLM is equipped with a single yet powerful tool: `jump`, which is realized through a language server.

Most existing agents rely on test-time scaling applied directly to pretrained LLMs (Liu et al., 2023; Chen et al., 2025; Schmidgall et al., 2025). In software engineering (SWE) tasks, tool usage is essential rather than optional: real-world repositories are far larger than the context window of current LLMs, making it impossible to process an entire codebase in a single forward pass. Agents must therefore iteratively invoke tools to retrieve partial information from the repository and interleave natural-language reasoning with tool calls.

However, mainstream LLMs are rarely exposed to such agentic interaction patterns during pretraining and typically acquire tool usage only through few-shot prompting. Such in-context demonstrations are insufficient for learning complex multi-step tool-chaining behaviors, especially under limited context windows. Moreover, because tool definition spaces are effectively unbounded, pretrained models cannot fully internalize their semantics without post-training. To mitigate these issues, post-training paradigms such as Supervised Finetuning (SFT) (Ma et al., 2025) and Reinforcement Learning with Verifiable Rewards (RLVR) (Yu et al., 2025a; Yue et al., 2025) have been applied, with promising results in domains including retrieval agents (Jin et al., 2025), GUI agents (Hong et al., 2024), and math agents (Yan et al., 2025).

[1]School of Computer Science, Peking University [2]Zhongguancun Academy. Correspondence to: Yitong Duan <duanyitong@zgci.ac.cn>, Yunfang Wu <wuyf@pku.edu.cn>.

Submitted to International Conference on Machine Learning, 2026.

Directly training an agent to fix software issues, however, remains difficult. A single bug often admits multiple valid patches, making string-level evaluation unreliable. The only precise evaluation method requires executing candidate patches inside a dedicated Docker environment for each repository (Luo et al., 2025), which is prohibitively expensive. To make training more tractable, we adopt a simplified yet widely generalizable assignment: **issue localization**. Prior work shows that a software issue becomes substantially easier to resolve once the relevant functions and files are correctly identified (Chen et al., 2025; Ma et al., 2025; Xia et al., 2024; Jiang et al., 2025). Since modern OSS repositories contain a significant amount of code—far beyond any LLM's context window—localization drastically reduces the search space and improves downstream solvability. Crucially, localization outputs a discrete set of paths, enabling verifiable, string-level evaluation that is compatible with scalable training frameworks such as SFT and RLVR.

Existing localization agents (Ma et al., 2025; Chen et al., 2025; He et al., 2025) typically rely on multiple tools, including `SearchClass`, `SearchMethods`, and `GetImports`. Although effective to some extent, these tools considers high-level abstractions (classes, function, etc) of programing languages, which do not reflect how code actually executes. High-level abstractions, such as classes or inheritance, disappear after compilation, leaving only sequential execution and `jump` operations. Since modern LLMs already excel at modeling sequential dependencies, we focus on enhancing their ability to `jump` across the repository—that is, to follow and inspect the source definition of symbols as they appear in execution. To this end, we introduce a single, structurally grounded tool: `jump`, which retrieves the precise definition of a given symbol. Details of this tool are provided in Sec. 3.3.

Our main contributions are threefold: (1) We propose the first repo-level localization agent trained on reinforcement learning directly from the pretrained model, regardless of distillation from a close-source model. (2) We design a repository-navigation agent that operates by performing realistic `jump` operations aligned with actual execution semantics. (3) We demonstrate that one unified tool significantly improves efficiency and controllability compared to multi-tool pipelines.

2. Related Works

2.1. Agentic Training

LLM agents are promising methods to equip models with complex tools while reasoning (Li et al., 2024; Huang et al., 2024; Guo et al., 2024b). However, because most pretrained LLMs are trained on texts only and developers can define any tools, most tools are out-of-domain (OOD) for LLMs. Even for the most powerful models, failures often happen when calling the new-defined tools due to wrong calling format or failed parameter parsing. Thus, training a LLM to master new-defined tool is critical for LLM agents. Intuitively, the tool-calling trajectories can be generated by a more powerful LLM, and such trajectories can be used to train a student model via supervised finetuning (SFT) (Chen et al., 2025). However, this pipeline requires a stronger teacher model which has capability to master the tool. Recently, more methods have emerged with no teacher-model required. Rejected-sampled finetuning (RFT) (Ahn et al., 2024) utilizes generated trajectories of the agent itself via multiple rollouts. Agentic RL (Jin et al., 2025) is an on-policy RLVR methods requiring only the result for verifiying trajectories. Such training methods yield remarkable results when the tools are search engines (Jin et al., 2025), python executer (Jimenez et al., 2023), calculator (Yan et al., 2025), and visual models (Gupta & Kembhavi, 2023).

2.2. Software Engineering Agents

The introduction of SWE-bench (Jimenez et al., 2023; Yang et al., 2024b) has motivated a range of agentic pipelines for software engineering (SWE) tasks. Among them, SWE-AGENT (Yang et al., 2024a) and OPENHANDS (Wang et al., 2025a) are widely adopted frameworks that equip agents with tools for interacting with computing environments. Workflow-based methods such as Agentless (Xia et al., 2024) decompose issue resolution into localization, repair, and validation subproblems. Chen et al. (2025) builds the respository as a graph and applied graph-level searching tools for localization, and Wang et al. (2025a) furthermore integrated commit history as agent memory. RepoLens (Wang et al., 2025b) equip conceptual information of the respository to enable repo-level understanding. These pipelines are training-free, compatible with closed-source language models, and yield competitive results.

To enable task-specific training, DEEPSWE (Luo et al., 2025) and SWE-SWISS (He et al., 2025) employ reinforcement learning and achieve strong performance. However, end-to-end training remains costly because patch evaluation requires executing Docker environments across numerous repositories. Consequently, issue localization has emerged as a computationally efficient alternative, aiming to identify faulty components—at file or function level—rather than generating full patches.

Recent localization agents include LOCAGENT (Chen et al., 2025) and COSIL (Jiang et al., 2025), which model codebases as graphs and integrates them into LLMs, and ORCALOCA (Yu et al., 2025b), which enhances efficiency through priority scheduling, action decomposition, and context pruning. From an open-source perspective, RE-

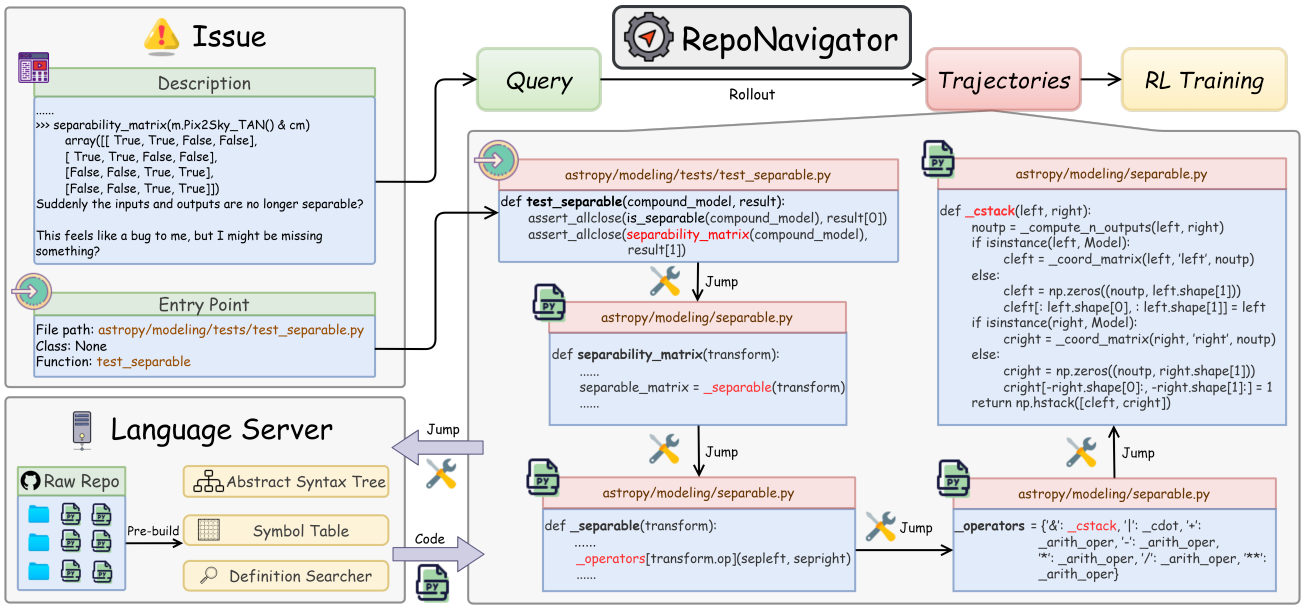

Figure 2. Overview of our RepoNavigator. During the rollout phrase, the agent can call the `jump` tool, and the language server will return the definition code of the symbol. This process is trained by reinforcement learning.

POSEARCHER (Ma et al., 2025), trained with distillation and RL on the Qwen model family (Team, 2024), represents a notable advancement.

Nevertheless, prior agents overlook the structural relations within repositories—where modules, classes, and functions are cross-referenced across files—and typically rely on multiple search tools for symbol definition retrieval, amplifying error propagation (see Sec. 3). In contrast, we employ a single execution-logic-focused tool, reducing usage complexity. Finally, our approach constitutes the first localization agent trained directly from pretrained models, without relying on distillation-based supervised finetuning, a crucial stage in both RepoSearcher (Ma et al., 2025) and LocAgent (Chen et al., 2025).

3. Method

We present **RepoNavigator**, a reinforcement-learning agent for repository-level issue localization. The method consists of three components: (1) a unified tool to retrieve the definition of any symbols in a given file, (2) a reasoning–action agent loop that alternates between natural-language reasoning and tool invocation, and (3) a GRPO-based RL algorithm for optimizing long-horizon tool-augmented trajectories. Below we provide the formal problem setting and the detailed method.

3.1. Problem Formulation

Given a repository $\mathcal{R} = \{f_1, \ldots, f_N\}$ and an issue description q, the goal is to output relevant code regions $Y^* = \{(f_i, g_{i,j})\}$, where $g_{i,j}$ denotes a function or code span in file f_i. At each step t, the agent produces a optional reasoning step r_t, a tool call a_t, and receives the observation o_t, forming a trajectory $\tau = \{(r_t, a_t, o_t)\}_{t=1}^{T}$. After termination, a final prediction $\hat{Y}$ is scored by a reward $R(\hat{Y}, Y^*)$. The objective is $\max_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$.

3.2. Agent Architecture

RepoNavigator uses a *single-tool* design to avoid multi-tool orchestration overhead. At each step the policy π_θ decides whether to continue reasoning or to emit a JSON-formatted tool call, while a symbol and its corresponding file are parsed to the tool. The agent receives structured observations (code snippets or error messages), then continues reasoning until termination. The loop is *reason* $\rightarrow$ *act* $\rightarrow$ *observe*.

3.3. Jump: Symbol Resolution

Language servers resolve the definition of a Python symbol through a deterministic static analysis pipeline that approximates Python's runtime name-binding semantics. Given a symbol occurrence s at source location ℓ, Pyright computes a resolution mapping

$$\mathcal{R}(s, \ell) \rightarrow \{(f_i, p_i)\}, \tag{1}$$

where each pair (f_i, p_i) denotes a file path and a source position corresponding to a valid definition site of s. In practice, we use `file_path` and `symbol` to resolve ℓ. If we have multiple symbols with the same name exist in the same code snippet, we additionally parse an `index` to the tool, which allows for accurate resolution of ℓ.

Syntactic Analysis In this process, the source file is parsed into an abstract syntax tree (AST). The syntactic role of s (e.g., name, attribute access, or call expression) determines the subsequent resolution strategy. For attribute expressions $a.b$, Pyright treats a as a receiver expression whose type must be inferred prior to member lookup.

Lexical Scope Resolution For a name symbol x, candidate definitions are searched along a scope chain

$$\mathcal{S} = \{\text{local}, \text{enclosing}, \text{module}, \text{builtins}\}, \tag{2}$$

following Python's LEGB rule. Each scope maintains a symbol table mapping identifiers to defining AST nodes.

Static Type Inference . For attribute symbols, it computes a (possibly union-valued) type $T(a)$ for the receiver expression a using type annotations, assignment flow analysis, function return types, and stub files (`.pyi`). Member resolution is then defined as

$$\text{resolve}(a.b) = \bigcup_{t \in T(a)} \text{lookup}(b, \text{MRO}(t)),$$

where $\text{MRO}(t)$ denotes the method resolution order of type t.

Import Dependency Graph For cross-file resolution, import dependency graph that statically emulates Python's module loading semantics is built. Import statements introduce bindings that map local symbols to exported symbols of target modules, including re-exports and `__all__`-based filtering. Resolution may therefore traverse multiple modules before reaching a concrete definition.

3.4. Reasoning–Action Loop

Given history $h_t = (q, o_{1:t-1}, a_{1:t-1})$, the agent samples either a natural-language reasoning step $r_t \sim \pi_\theta(\cdot|h_t)$ or a structured tool call $a_t \sim \pi_\theta(\cdot|h_t)$. Tool calls must satisfy a JSON grammar enforced via constrained decoding. The loop continues until the agent outputs its final localization $\hat{Y}$.

3.5. Reinforcement Learning

We apply reinforcement learning with verifiable rewards to train the agent directly from the pretrained model, with no teacher model required. In practice, we apply Group Reference Policy Optimization (GRPO), which has the loss function:

$$\mathcal{L}^{\text{GRPO}}(\theta) = \mathbb{E}_{(s_t,a_t)\sim\pi_{\theta_{\text{old}}}} \Bigg[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t - \beta\, D_{\text{KL}}\left(\pi_{\theta_{\text{old}}}(\cdot|s_t) \| \pi_\theta(\cdot|s_t)\right)\Bigg] \tag{3}$$

where the first term is the standard policy gradient objective with an estimated advantage function $\hat{A}_t$, which promotes actions that lead to higher-than-expected returns. The second term is a Kullback-Leibler (KL) divergence penalty, scaled by a coefficient β, which acts as a trust region, preventing the updated policy π_θ from moving too far from the previous policy $\pi_{\theta_{\text{old}}}$. This formulation ensures stable and consistent policy improvement by balancing reward maximization with behavioral consistency.

The reward of GRPO process is calculated as:

$$R(\hat{Y}, Y^*, \tau) = \text{DICE}(\hat{Y}, Y^*) + \text{S}(\tau) \tag{4}$$

Dice is a common metric for set-level comparison, for set $\hat{Y}$ and set Y^*

$$\text{DICE}(\hat{Y}, Y^*) = \frac{2 \times |\hat{Y} \cap Y^*|}{|\hat{Y}| + |Y^*|} \tag{5}$$

and $S(\tau)$ is the success rate of tool-calling extracted from τ. We consider the tool-call to be failed when the format is incorrect, or the symbol parsed does not exist, or for any other reason that causes the tool to quit unexpectedly.

4. Experiment

4.1. Experimnent Setup

Datasets We extract valid samples from SWE-smith (Yang et al., 2025b) to form the training set. We apply Qwen2.5-7B-Instruct with RepoNavigator to sample each data for 16 times. A sample is abandoned if all 16 scores are zero. For validation, we test our method on SWE-bench-verified (Jimenez et al., 2023), which is a human-verified subset of SWE-bench. We additionally test our method on a subset of SWE-bench-pro (Yang et al., 2025b) (which is a new and more difficult benchmark) for generalization. For ground-truth locations, we directly use the locations in golden patches. All datasets are open-source and are built on real-world github issues.

Metrics Previous works (Chen et al., 2025; Ma et al., 2025) applied recall and precision as metrics. However, because the predicted locations and ground-truth locations are sets of strings, recall and precision singularly can not reflect the performance fairly. Thus, we utilize Sample-F1

Table 1. Comparison of different agent pipelines on function-level and file-level Dice/IoU metrics. We use Qwen2.5-Instruct series as our base model. **Bold numbers** denote the best performance among same-size models; underline numbers denote the best training-free performance among same-size models; yellow background illustrates training-free RepoNavigator; blue background illustrates RepoNavigator trained with GRPO.

Agent Pipeline	Model	Function-level				File-level			
		Recall	Precision	Sample-F1	IoU	Recall	Precision	Sample-F1	IoU
Close-source Models									
RepoSearcher	Claude3.7-Sonnet	**66.80**	19.90	28.30	17.89	**89.71**	21.04	33.15	20.67
RepoNavigator	Claude3.7-Sonnet	31.03	34.43	31.72	30.22	72.26	75.95	73.01	71.37
RepoNavigator	GPT5-chat	30.42	34.56	31.17	29.67	58.17	61.87	58.88	57.33
RepoNavigator	Claude4.5-Sonnet	43.97	**45.76**	**43.62**	**41.31**	80.68	**81.92**	**79.94**	**77.49**
Qwen2.5-7B									
Locagent	Training Free	17.62	11.71	12.71	10.31	60.96	34.88	40.67	33.33
CoSIL	Training Free	29.30	8.98	12.90	8.07	70.12	17.90	27.39	17.42
Agentless	Training Free	24.92	12.93	15.31	11.74	63.01	19.32	27.82	18.85
Orcaloca	Training Free	27.70	20.29	21.70	17.92	48.04	48.65	47.36	45.77
RepoSearcher	Distillation+GRPO	**63.26**	19.24	27.37	17.59	**84.11**	19.97	31.64	19.57
RepoNavigator	Training Free	15.89	17.46	16.19	15.46	42.36	43.23	42.12	40.97
RepoNavigator	GRPO	26.69	**30.34**	**27.49**	**26.43**	50.62	**53.83**	**51.63**	**50.62**
Qwen2.5-14B									
Locagent	Training Free	35.62	13.32	17.71	12.32	71.42	31.66	40.77	30.64
CoSIL	Training Free	**48.61**	13.40	19.81	12.12	**78.35**	18.10	28.79	17.72
Agentless	Training Free	25.20	14.30	16.14	12.28	75.65	19.76	29.88	19.30
Orcaloca	Training Free	29.92	20.98	22.77	18.92	52.17	52.15	50.93	48.72
RepoSearcher	Training Free	26.13	11.96	14.35	10.60	74.77	18.80	28.79	18.15
RepoNavigator	Training Free	27.96	25.77	25.58	23.00	59.00	56.68	56.39	53.74
RepoNavigator	GRPO	31.02	**30.08**	**29.23**	**26.84**	61.60	**58.97**	**58.90**	**56.36**
Qwen2.5-32B									
Locagent	Training Free	46.79	16.29	21.48	14.18	79.39	34.18	44.18	33.24
CoSIL	Training Free	55.38	14.85	22.11	13.52	83.50	19.34	30.77	18.93
Agentless	Training Free	40.79	24.07	27.33	22.08	78.93	25.60	35.38	24.96
Orcaloca	Training Free	39.14	25.59	28.72	22.89	59.57	59.51	58.11	55.62
RepoSearcher	Distillation+GRPO	**69.50**	20.29	29.11	18.23	**89.33**	20.27	32.93	20.35
RepoNavigator	Training Free	28.11	28.19	27.12	25.16	63.05	62.75	61.67	59.28
RepoNavigator	GRPO	33.71	**37.19**	**34.09**	**32.30**	67.29	**70.76**	**67.75**	**65.75**

(which is the averaged score of per-sample F1 values) and IoU (intersection out of union) as our core metrics. At the same time, we also present the recall and precision scores to align with previous methods, although they do not reflect the methods' performance fairly.

Training For the 7B model, we conduct GRPO with 8 Tesla-A100-80G GPUs. For the 14B and 32B model, we train it with 16 Tesla-A100-80G GPUs. We apply verl (Shen, 2024) as the training framework, and we apply vLLM (Kwon et al., 2023) as the inference engine. We train the model for 1 epoch, while the training batch size is fixed to 128 on 4k training samples filtered from SWE-smith, with maximum prompt length and max response length both set to 10240. Additionally, we rollout 8 times for each sample, and the temperature is set to 1.0 to encourage exploration. We use greedy decoding in the inference stage to ensure stable performance. More implementation details are provided in Appendix. B.

4.2. Effectiveness

Baselines We compare our method against Locagent (Chen et al., 2025), CoSIL (Jiang et al., 2025), Agent-

Table 2. Comparison of different agent pipelines on function-level and file-level metrics on SWE-bench_Pro for generalization. **Bold numbers** denote the best performance among same-size models; underline numbers denote the best training-free performance among same-size models; yellow background illustrates training-free RepoNavigator; blue background illustrates RepoNavigator trained with GRPO.

Agent Pipeline	Model	Function-level				File-level			
		Recall	Precision	Sample-F1	IoU	Recall	Precision	Sample-F1	IoU
Qwen2.5-7B									
LocAgent	Training Free	1.01	0.02	0.65	0.40	12.16	0.17	10.81	8.93
CoSIL	Training Free	8.64	3.33	4.58	2.87	26.64	8.47	12.11	7.70
Agentless	Training Free	**12.82**	6.94	8.05	5.73	**39.41**	13.15	18.89	12.35
RepoSearcher	Training Free	1.07	0.93	0.97	0.86	4.91	1.64	2.30	1.63
RepoNavigator	Training Free	9.84	14.65	10.67	9.20	30.50	37.24	31.86	28.82
RepoNavigator	GRPO	12.33	**21.26**	**14.29**	**12.02**	36.36	**48.13**	**39.74**	**36.36**
Qwen2.5-14B									
LocAgent	Training Free	6.22	0.13	3.65	2.65	15.58	0.21	11.69	9.53
CoSIL	Training Free	10.73	4.67	5.96	3.94	34.31	9.97	14.81	9.30
Agentless	Training Free	10.49	6.75	7.41	5.28	41.42	13.42	19.02	12.37
RepoSearcher	Training Free	2.79	1.38	1.69	1.14	17.37	5.17	7.60	4.84
RepoNavigator	Training Free	14.36	19.74	15.27	12.00	43.57	54.52	46.06	41.07
RepoNavigator	GRPO	**16.05**	**25.25**	**18.06**	**14.58**	**46.85**	**58.64**	**49.72**	**45.14**
Qwen2.5-32B									
LocAgent	Training Free	8.72	0.17	4.30	2.90	25.73	0.38	19.77	16.50
CoSIL	Training Free	15.00	6.35	8.14	5.21	45.37	13.04	19.42	12.36
Agentless	Training Free	11.08	7.31	7.98	5.80	43.07	13.89	20.07	13.11
RepoSearcher	Training Free	2.00	1.29	1.45	1.00	13.51	3.43	5.31	3.24
RepoNavigator	Training Free	13.96	20.25	15.36	12.87	50.24	63.24	53.48	48.50
RepoNavigator	GRPO	**18.13**	**29.44**	**20.72**	**17.16**	**53.49**	**68.69**	**57.57**	**52.44**

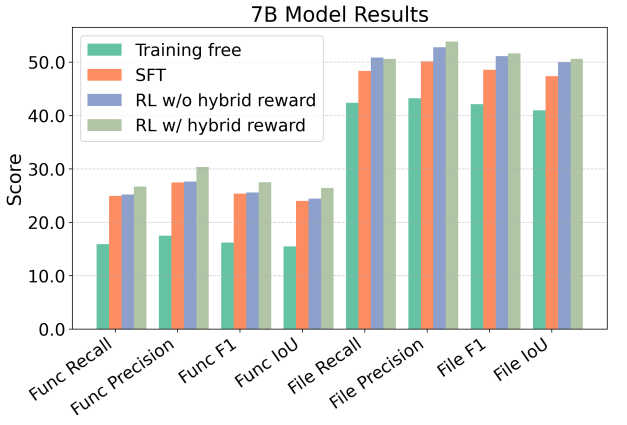

Figure 3. Ablation study: comparison between RepoNavigator with training free, RFT, GRPO with pure outcome and hybrid reward on Qwen2.5-7B-Instruct.

less (Xia et al., 2024), Orcaloca (Yu et al., 2025b), and RepoSearcher (Ma et al., 2025). Detailed explaination of baseline methods are presented in Appendix. A.

Results As illustrated in Table. 1, on balanced metrics (S-F1 and IoU) for both function-level and file-level localization, our method surpasses all baseline methods with the same model size. Moreover, if we train RepoNavigator with GRPO, our 7B model surpasses 14B baselines, and our 14B model surpasses 32B baselines on S-F1 and IoU. This contributes to the validness of RepoNavigator furthermore. Although some baselines have higher recall score significantly lower precision score than RepoNavigator, and result in lower S-F1 and IoU. This indicates that RepoNavigator behaves more conservatively and generates less wrong locations. For 14B and 32B models, RepoNavigator achieves SOTA among all training-free methods. This implies that the tool we implement is effective and promising, and our single tool pipeline is better than previous multiple tools pipelines.

Compared with RepoSearcher, which is distilled from claude-3.7-sonnet (Anthropic, 2025) and reinforced by

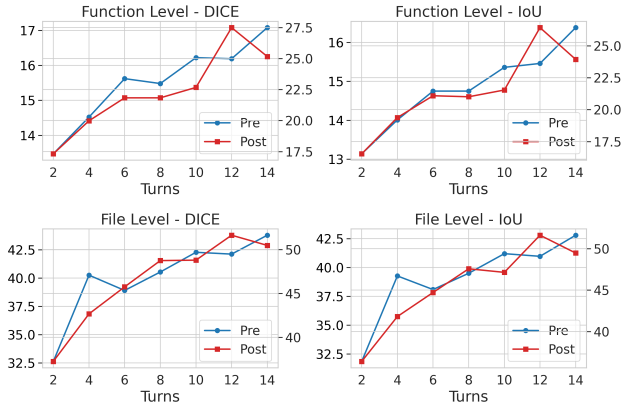

Figure 4. Scaling law of tool-calling, where *Pre* and *Post* denote the corresponding metric before and after the RL training.

GRPO, trained RepoNavigator outperforms it on all metrices except recall. Moreover, we found that our training-free method outperforms RepoSearcher for 14B models. This is probably due to the simplified tool we integrate to the agent (see Sec. 5 for more details).

To assess the generalizability of RepoNavigator, we present its performance on Python samples from the SWE-bench-Pro dataset (Yang et al., 2025b) in Table 2. The results on this dataset are consistent with those observed on SWE-bench_Verified. While we cannot fully exclude the potential influence of data leakage in SWE-bench_Verified, we can make a stronger claim regarding SWE-bench_Pro, as it was released after the publication of the Qwen2.5 series.

4.3. Training Strategy Comparison

To explore the capability of GRPO on agentic training, we compare GRPO against RFT-only and RFT+GRPO. As presented in Fig. 3, directly training with GRPO outperformes RFT-only and RFT+GRPO. Moreover, although RFT has accetable performance, the more steps RFT proceeds, the less improvement GRPO makes after the cold start. This conclusion contradicts with previous SWE agents trained with RL (Ma et al., 2025), however, it aligns with the broader field of reinforcement learning, where RFT and SFT (as a cold start) is effective only when the pretrained model is not strong enough (Guo et al., 2024a). When the pretrained model is strong enough and data is high-quality, directly training a model with RL is better than training after SFT (RFT) as its cold start.

We also remove the success rate in the reward function for ablation. As presented in Fig. 3, reinforcement learning with hybrid reward (with tool-calling success rate) has higher performance than pure outcome reward (without tool-calling success rate). This indicates that learning to correctly call tools is vital in agentic learning.

Agent Pipeline	Func-IoU(%)	Resolved(%)
Agentless	5.28	10.12
LocAgent	2.65	13.01
RepoNavigator	12.00	14.74
RepoNavigator+RL	14.58	15.03

Table 3. We use Qwen2.5-14B-Instruct as the localization model, and use Qwen2.5-32B-Instruct as the repair model on SWE-bench_Verified.

4.4. Scaling Law of Tool-Calling

To assess the significance of tool-calling in RepoNavigator, we varied the maximum number of tool-calling turns and reported the results in Fig. 4.2. As shown in the figure, allowing more tool-calling turns consistently leads to improved performance for RepoNavigator, both before and after reinforcement learning (RL) training. In other words, these results empirically validate the scaling law of tool-calling in this context.

4.5. Influence on Issue Resolution

To evaluate the impact of different localization results on the final issue resolution performance, we test RepoNavigator against baselines on SWE-bench_Verified. We directly apply the repairing phrase of Agentless while replacing its localization front-end with other methods. Table.3 illustrates the results. Compared with baselines, RepoNavigator has the highest performance on issue resolution, while reinforcement learning improves its performance furthermore.

5. Discussion: Building Less yet More Capable Tools

In this section, we analyze the logic behind RepoNavigator: building less tools with more powerful and more ensembled functions is more effective than building multiple task-specific tools.

5.1. Impact on the Action Space of Agents

Let the total number of available tools be denoted as k. When only a single tool—specifically the `jump` tool—is retained, the system's structural relations become simpler, as both the action space and the observation space are restricted to what this tool can access. In this case, the set of possible actions and observable elements is smaller than when multiple tools are available. This reduction is generally beneficial, since additional tools often introduce new and unfamiliar interfaces that large language models have not been exposed to during pretraining, potentially increasing the likelihood of errors.

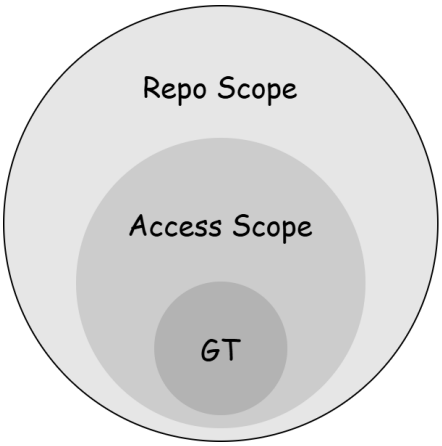

Figure 5. Venn graph illustrating access scope of `jump`. Compared with the repository scope, the access scope has a much higher IoU with the groundtruth set.

5.2. Impact on Tool-Calling Success Rate

For a given process in issue localization (for instance, checking the code snippet of a function), let the success probability of the i-th call be p_i. For a task that requires k sequential tool invocations, the overall success rate can be expressed as

$$P_{\text{succ}}(k) = \prod_{i=1}^{k} p_i. \tag{6}$$

Since each step introduces an additional potential point of failure, the cumulative success rate typically decreases as the number of required tool calls increases. Therefore, in general, completing a task with a single, more versatile tool tends to be more reliable than relying on multiple narrow-scope tools executed in sequence.

5.3. Impact on the Prediction Space

The access scope of a tool is defined as the complete set of files, symbols, and other resources that the tool can access within a repository. For a `jump` tool that navigates to symbol definitions, its access scope can be obtained by starting from a given entry point and recursively resolving all referenced symbols until no new definitions can be reached. Apparently, its access scope is significantly smaller than the full repository scope. Consequently, when computing the Intersection over Union (IoU) between the prediction set and the groundtruth set, using the `jump` tool results in a higher IoU, as depicted in Fig. 5. On the other hand, applying multiple repo-level retrivel tools results in the access scope equal to the whole repository scope.

When we start from the entry point and repeatedly apply `jump`—which retrieves the definition of each referenced symbol—we effectively traverse all symbols that are semantically activated by that entry point. Because every location that contributes to the issue must lie on some dependency path originating from the entry point, it is necessarily reachable through this recursive symbol-reference expansion. Therefore, the final access scope produced by exhaustive `jump` traversal is guaranteed to contain all locations that must be modified to resolve the issue.

Jump	GetClass	GetFunc	GetStruc	IoU
✓	✓	✓	✓	13.71
✓	✓	✓	✗	21.44
✓	✗	✗	✓	24.00
✓	✗	✗	✗	24.28

Table 4. We change the tool set of RepoNavigator and present the function-level IoU (%) on Qwen2.5-7B-Instruct. Apparently, excessive tools do not boost RepoNavigator's performance.

5.4. Verification

To further verify this proposal, we change the tool set of RepoNavigator and conduct RL training with only the outcome reward. We add excessive tools which were frequently used in previous works (Chen et al., 2025; Ma et al., 2025; Jiang et al., 2025) and present the result in Table. 4. *GetClass/GetFunc* takes a class/function name as input and outputs the class/function definition. *GetStruc* takes no input and outputs the repository's structure. The results clearly implies that additional tools do not increase model's performance. This inspires researchers to develop **less but more capable tools**.

6. Conclusion

In this work, we introduced RepoNavigator, a repository-level issue localization agent that departs from existing multi-tool paradigms by leveraging a single, more-capable `jump` tool for symbol resolution. This unified design faithfully reflects real code execution flow while significantly reducing the complexity and brittleness of multi-step tool chaining. Through tool-integrated GRPO, RepoNavigator learns to reason, invoke tools, and refine its predictions in a closed-loop manner, enabling end-to-end optimization without relying on closed-source teacher models or distillation.

Extensive experiments across SWE-bench-Verified and SWE-bench-Pro demonstrate that RepoNavigator achieves state-of-the-art localization performance. We theoretically analyze the results, confirming that a single powerful tool, jointly optimized with reinforcement learning, can provide stronger robustness and more reliable multi-step reasoning than previous frameworks relying on multiple narrowly scoped tools.

Our findings highlight the importance of aligning agent tooling with real execution structure, and show that efficient reasoning-tool co-training can unlock substantial gains even for medium-sized open-source models. Future work will explore extending RepoNavigator from Python to more programming languages.

References

Ahn, J., Verma, R., Lou, R., Liu, D., Zhang, R., and Yin, W. Large language models for mathematical reasoning: Progresses and challenges. *arXiv preprint arXiv:2402.00157*, 2024.

Anthropic. Claude 3.7 sonnet and claude code. `https://www.anthropic.com/news/claude-3-7-sonnet`, February 2025. data: 2025-11-18.

Chen, Z., Tang, R., Deng, G., Wu, F., Wu, J., Jiang, Z., Prasanna, V., Cohan, A., and Wang, X. LocAgent: Graph-guided LLM agents for code localization. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8697–8727, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.426. URL `https://aclanthology.org/2025.acl-long.426/`.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y., et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024a.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., and Zhang, X. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024b.

Gupta, T. and Kembhavi, A. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 14953–14962, 2023.

He, Z., Yang, Q., Sheng, W., Zhong, X., Zhang, K., An, C., Shi, W., Cai, T., He, D., Chen, J., and Xu, J. Swe-swiss: A multi-task fine-tuning and rl recipe for high-performance issue resolution. https://github.com/zhenyuhe00/SWE-Swiss, 2025. Notion Blog.

Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Dong, Y., Ding, M., et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 14281–14290, 2024.

Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., Wang, Y., Tang, R., and Chen, E. Understanding the planning of llm agents: A survey. *arXiv preprint arXiv:2402.02716*, 2024.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Jiang, Z., Ren, X., Yan, M., Jiang, W., Li, Y., and Liu, Z. Cosil: Software issue localization via llm-driven code repository graph searching. *arXiv preprint arXiv:2503.22424*, 2025.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., and Han, J. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.

Li, Y., Wen, H., Wang, W., Li, X., Yuan, Y., Liu, G., Liu, J., Xu, W., Wang, X., Sun, Y., et al. Personal llm agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*, 2024.

Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Liu, Z., Zhang, Y., Li, P., Liu, Y., and Yang, D. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. *arXiv preprint arXiv:2310.02170*, 2023.

Lu, J., Holleis, T., Zhang, Y., Aumayer, B., Nan, F., Bai, F., Ma, S., Ma, S., Li, M., Yin, G., et al. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. *arXiv preprint arXiv:2408.04682*, 2024.

Luo, M., Jain, N., Singh, J., Tan, S., Patel, A., Wu, Q., Ariyak, A., Cai, C., Tarun Venkat, S. Z., Athiwaratkun, B., Roongta, M., Zhang, C., Li, L. E., Popa, R. A., Sen, K., and Stoica, I. Deepswe: Training a state-of-the-art coding agent from scratch by scaling rl. `https://pretty-radio-b75.notion.site/DeepSWE-Training-a-Fully-Open-sourced-State-of-the-Art-Coding-Agent-by-Scaling-RL-22281902`. 2025. Notion Blog.

Ma, Z., Peng, C., Zeng, Q., Gao, P., Zou, Y., and Xie, B. Tool-integrated reinforcement learning for repo deep search, 2025. URL `https://arxiv.org/abs/2508.03012`.

Schmidgall, S., Su, Y., Wang, Z., Sun, X., Wu, J., Yu, X., Liu, J., Moor, M., Liu, Z., and Barsoum, E. Agent laboratory: Using llm agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025.

Shen, Z. Llm with tools: A survey. *arXiv preprint arXiv:2409.18807*, 2024.

Team, Q. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H. H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., and Neubig, G. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL `https://openreview.net/forum?id=OJd3ayDDoF`.

Wang, Y., Mao, W., Wang, C., Zhou, Z., Zhou, Y., Zhao, W., Lou, Y., and Peng, X. Extracting conceptual knowledge to locate software issues. *arXiv preprint arXiv:2509.21427*, 2025b.

Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

Yan, Y., Wang, S., Huo, J., Yu, P. S., Hu, X., and Wen, Q. Mathagent: Leveraging a mixture-of-math-agent framework for real-world multimodal mathematical error detection. *arXiv preprint arXiv:2503.18132*, 2025.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. R., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a. URL `https://arxiv.org/abs/2405.15793`.

Yang, J., Jimenez, C. E., Zhang, A. L., Lieret, K., Yang, J., Wu, X., Press, O., Muennighoff, N., Synnaeve, G., Narasimhan, K. R., et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024b.

Yang, J., Lieret, K., Jimenez, C. E., Wettig, A., Khandpur, K., Zhang, Y., Hui, B., Press, O., Schmidt, L., and Yang, D. Swe-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025b.

Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025a.

Yu, Z., Zhang, H., Zhao, Y., Huang, H., Yao, M., Ding, K., and Zhao, J. Orcaloca: An llm agent framework for software issue localization, 2025b. URL `https://arxiv.org/abs/2502.00350`.

Yuan, S., Song, K., Chen, J., Tan, X., Shen, Y., Kan, R., Li, D., and Yang, D. Easytool: Enhancing llm-based agents with concise tool instruction. *arXiv preprint arXiv:2401.06201*, 2024.

Yue, Y., Yuan, Y., Yu, Q., Zuo, X., Zhu, R., Xu, W., Chen, J., Wang, C., Fan, T., Du, Z., et al. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.

A. Detailed Illustration of Baselines

Agentless Agentless (Xia et al., 2024) is a workflow for issue localization. First, it identifies suspicious files in the repository. Second, relevant classes and functions are detected. Third, precise locations for edit are given by LLMs based on the classes and functions.

CoSIL CoSIL (Jiang et al., 2025) is an agent which first conduct file-level localization and then conduct function-level localization. CoSIL dynamically constructs call graphs of modules (class, functions) during the repo-level searching process, and applies context pruning to effectively reduce the searching scope.

LocAgent LocAgent (Chen et al., 2025) is almost a fully-automatic LLM agent besides its planning prompt concatenated into the context at the beginning of the searching process. It builds the whole repository into a direct heterogeneous graph, whose nodes are files, classes, and functions. Additionally, edges are built by dependencies such as imports and invocations. Multiple graph-level searching tools are equipped to the LLM for multi-hop reasoning.

RepoSearcher RepoSearcher (Ma et al., 2025) is an agent that first conducts file-level localization and then function-level localization, which aligns with CoSIL. RepoSearcher introduced the first training framework *ToolTrain* for localization agents, which is composed of distilling from a close-source model (Claude3.7-Sonnet in RepoSeacher) as warmup and reinforcement learning to further enhance the performance.

Ours Compared with all baselines, we are the first fully-automatic LLM agent, with no fixed workflow and no planetary prompt, and we are the first method trained directly from pretrained open-source LLMs without a close-source teacher model. Lastly, we only integrate a single yet powerful tool to the agent, which reduces compounding error and narrows the access scope of the agent.

B. Experimental Details

Hyperparameters We set clip_ratio_low to 0.2, clip_ratio_high to 0.8, learning rate to 10^{-6}, training_batch_size to 128,training temperature to 1.0, maximum tool-calling times to 12, and max_response_length to 10240.

Metrics Given the set of predicted locations (ether file-level or function-level) $\hat{Y}$, and the set of groundtruth locations Y^*, the aforementioned metrics are calculated as:

$$\text{Recall} = \frac{|\hat{Y} \cap Y^*|}{|Y^*|} \tag{7}$$

Jump	GetClass	GetFunc	GetStruc	Recall	Precision	F1	IoU	Recall	Precision	F1	IoU
✓	✓	✓	✓	14.28	15.44	14.40	13.71	35.78	36.76	35.59	34.55
✓	✓	✓	✗	22.60	25.02	22.80	21.44	48.49	50.13	48.52	47.17
✓	✗	✗	✓	24.64	27.48	25.05	24.00	53.48	55.76	53.68	52.69
✓	✗	✗	✗	**25.11**	**29.16**	**25.75**	**24.28**	**55.81**	**58.71**	**56.32**	**54.89**

Table 5. We change the tool set of RepoNavigator and present the function-level IoU. Because the `jump` tool is already powerful enough for localization, excessive tools do not increase its performance.

$$\text{Precision} = \frac{|\hat{Y} \cap Y^*|}{|\hat{Y}|} \tag{8}$$

$$\text{Sample-F1} = \frac{2 \times |\hat{Y} \cap Y^*|}{|\hat{Y}| + |Y^*|} \tag{9}$$

$$\text{IoU} = \frac{|\hat{Y} \cap Y^*|}{|\hat{Y} \cup Y^*|} \tag{10}$$

In practice, when the prediction set $\hat{Y}$ is empty (for instance, total failure), we set recall, precision, sample-F1, and IoU to zero. We use the function-level localization result of different methods and apply the patch generation backend in Agentless (Xia et al., 2024) to generate patches. Resolved(%) denotes the percentage of samples that pass all test units after applying the patch.

Implementation When the response exceeds the maximum length, we clip and force the agent to stop, and we give zero as its score. When the agent exceeds the maximum tool-calling times (which is 12), we add **"You must not call tools anymore, and you must give the final answer"** to the tool's response. Most of the time, the agent will stop calling tools and generate the final response. If not, we force it to stop and give zero as its score. Note that when the maximum tool-calling times is not achieved and the final answer is generated, the agent loop will stop automatically. The aforementioned process is an automatic agentic framework, which allows the agent to explore in the environments with little constraints.

Preventing Data Leakage It is a widespread concern that data leakage at the pre-training phrase threatens the validity of post-training methods. Nevertheless, we exclude this concern by results in Tabel. 2. The SWE-bench_Pro dataset was published in 2025, while the Qwen2.5 series were published in 2024. Moreover, we exclude the samples in the training dataset if the repository also appears in SWE-bench_Verified or SWE-bench_Pro.

Language Server In practice, we apply a Python language server to extract the definition code corresponding to an invoked symbol within a repository. However, the presence of monkey patches—runtime modifications to the repository—and dynamic imports can degrade the performance of the language server, as its functionality relies on static analysis techniques such as abstract syntax trees and symbol tables. When such circumstances occur, the tool returns an error message indicating that the definition of the current symbol cannot be located due to unknown reasons. Nevertheless, in our empirical evaluation, we did not observe any instances of monkey patching or dynamic imports within the analyzed datasets.

C. Threats to Validity

Groundtruth Retrieval A limitation of our work lies in the extraction of groundtruth locations. We extract modified locations directly from the `gold_patch` in the datasets, which may ignore other patches that also resolve the issue. Our evaluation metrics do not take these correct alternatives into consideration. However, using golden patches is acceptable when comparing mutliple methods. If a method reveals golden locations (locations in golden patches), it undoubtedly contributes to the resolution of the issue, and the result in Table. 3 demonstrates this claim.

Language Limit Another limitation is that we only evaluate Python repositories in our experiments. This is because each language (C/C++, Java, etc.) has its unique language server, and we only succeed in implementing the language server of python. We will implement more language servers and validate our approach on more programing languages in the future.

D. Case Study

In this section, we present the full trajectory of RepoNavigator on *astropy_astropy-12907* from SWE-bench_Verified. We apply the default tool-calling prompt template of verl (Shen, 2024) and present an example. Noted, we do not present any process restrictions in our prompt, encouraging RepoNavigator to plan, call tools, and make decisions full-automatically. This is distinct with Agentless (which has a fixed workflow), LocAgent (which predefines a specific step-by-step workflow in its system prompt), CoSIL and RepoSearcher (which is half-automatic because some

forced steps are added to the workflow besides the automatic multi-turns tool-calling conversations).

Prompt

```
[system]
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{"type": "function", "function": {"name": "check", "description": "In the
specific file path, a symbol is referred and this tool can find
where the tool is defined.
For instance, in the first turn, file_path is the entry point of.",
"parameters": {"properties": {"symbol": {"description": "The symbol whose
definition code will be given to the agent.", "type": "string"}, "file_path":
{"description": "The relevant path to the file where the symbol is referred.",
"type": "string"}}, "required": ["symbol", "file_path"], "type": "object"}}}
</tools>

For each function call, return a json object with function name and arguments
within <tool_call></tool_call> XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>
[user]
You are given a codebase and an issue, you need to locate the files and
functions causing this issue.
You can call the tool to check the definition code of a symbol. You can only
check the symbol once for each turn.
The 'file_path' is the relevant path of where the symbol is called,
NOT where it is defined!
For instance, if 'classA.functionB' is what you want to check (which is called
in fileA.py), you should directly check 'functionB' in 'fileA.py'.

This is the issue:
[Problem Statement]


The entry file of the code base is:
[Relevant Path To Entry Point]
[Entry Point]


Your final answer should be all functions that should be modified, such as:
relevant/path/to/file1.py::func_name1,relevant/path/to/file2.py::func_name2,
...(a series of file::function pairs seperated by comma)
Please put your final answer inside \boxed{} only in the last turn.
You can only call the tool once each turn.
For instance:
{'name': 'check', 'arguments': {'symbol': 'symbol_to_be_checked', 'file_path':
'file_where_the_symbol_is_used'}}
```